

Redes recurrentes y convolucionales: pronóstico de temperatura con MeteoNet y clasificación de Fashion-MNIST

Daniel Sebastian Velasco Munar^{a,c}, Andrea Barraza^{a,c}

Andrés Moreno^{b,c}

^aEstudiantes de la Maestría en Inteligencia Artificial

^bProfesor Asistente, Departamento de Ingeniería de Sistemas

^cPontificia Universidad Javeriana, Bogotá, Colombia

Resumen—Este trabajo aplica redes neuronales recurrentes y convolucionales a dos problemas de aprendizaje profundo. En el primero se pronostican las siguientes 24 horas de temperatura en la estación 62548002 de MeteoNet, en la costa norte de Francia, a partir de las 168 horas previas de doce variables meteorológicas y de calendario. Una búsqueda por etapas de 18 configuraciones mostró que una red recurrente simple de 32 unidades y 2,232 parámetros supera a las celdas GRU y LSTM, que sobreajustan en pocas épocas. Sobre el 30 % final de la serie el modelo obtiene un error absoluto medio de 1.60 °C, 24.7 % menos que la mejor línea base, que repite la temperatura del día anterior, y le gana en los 24 horizontes. En el segundo se clasifican las diez clases de Fashion-MNIST con tres aproximaciones entrenadas bajo la misma partición, semilla, lotes y criterios de parada: una red convolucional diseñada desde cero, VGG16 con filtros de ImageNet y MobileNetV2 con aprendizaje por transferencia. VGG16 alcanza una exactitud de 93.19 %, MobileNetV2 de 92.43 % y la red propia de 89.04 %. Una prueba pareada muestra que la ventaja de VGG16 sobre MobileNetV2 es real pero pequeña, a cambio de ocho veces más tiempo de entrenamiento. Como extensión, EfficientNetV2-B0 alcanza 92.73 % y es la única red que no se distingue estadísticamente de VGG16, con 4.5 veces menos tiempo, por lo que resulta la opción más equilibrada. En ambos problemas la capacidad adicional no se tradujo en mejor generalización, y el error remanente se explica más por la información disponible que por la arquitectura.

Términos de indexación—Aprendizaje por transferencia, EfficientNetV2, Fashion-MNIST, MeteoNet, MobileNetV2, pronóstico de series de tiempo, redes neuronales convolucionales, redes neuronales recurrentes, VGG16.

I. INTRODUCCIÓN

LAS redes neuronales recurrentes (RNN) y las redes neuronales convolucionales (CNN) son las dos familias clásicas del aprendizaje profundo para datos con estructura. Las primeras procesan secuencias manteniendo un estado interno que resume el pasado [1], [2], [3]; las segundas aprovechan la localidad y la invariancia a las traslaciones de las imágenes mediante filtros compartidos [4]. Este informe presenta el Taller 1 del curso Aprendizaje Profundo, que pide aplicar cada

familia a un problema real y analizar con método qué funciona y qué no.

El Punto 1 construye un modelo recurrente para pronosticar la temperatura de una estación meteorológica de MeteoNet, el conjunto de datos abierto de Météo-France [5]. El Punto 2 compara tres formas de resolver la clasificación de Fashion-MNIST [6]: una CNN diseñada desde cero, una red que reutiliza los filtros de VGG16 [7] entrenados en ImageNet [8] y una adaptación de MobileNetV2 [9] por aprendizaje por transferencia; como extensión, repite la comparación con EfficientNetV2-B0 [33], una red moderna de la misma línea que MobileNetV2.

Tres decisiones atraviesan ambos puntos. Primero, toda selección de hiperparámetros se hace con un conjunto de validación y el conjunto de prueba (*test*) se usa una sola vez, al final. Segundo, cada resultado se lee frente a una referencia: líneas base sin aprendizaje en el Punto 1 y un protocolo común para las tres redes en el Punto 2, que es lo que permite una comparación justa. Tercero, la incertidumbre se hace explícita cuando las diferencias son pequeñas. Las contribuciones son: 1) una formulación documentada y sin fuga de información de la tarea de pronóstico a 24 h; 2) una búsqueda de hiperparámetros por etapas que muestra por qué la celda recurrente más simple es la adecuada para este problema; 3) una comparación controlada de tres aproximaciones convolucionales, extendida a una red moderna, en exactitud, costo e incertidumbre, con pruebas pareadas sobre el mismo conjunto de prueba, y 4) un análisis de errores que separa las limitaciones del modelo de las de los datos.

La Sección II resume el trabajo relacionado; las Secciones III y IV presentan el diseño experimental y los resultados de cada punto; la Sección V reúne las lecciones aprendidas y la Sección VI concluye. El código, los resultados y las figuras están en el repositorio del trabajo [10].

II. TRABAJO RELACIONADO

A. Pronóstico de temperatura con redes recurrentes

Las RNN simples [1] sufren el desvanecimiento del gradiente en secuencias largas [11], lo que motivó las celdas con compuertas LSTM [2] y GRU [3]; en tareas de modelado de

secuencias ambas suelen rendir de forma parecida [12]. En meteorología, estas redes se han usado para pronosticar la temperatura a partir de las observaciones de una estación: Karevan y Suykens [13] proponen una LSTM transductiva que da más peso a los ejemplos parecidos al punto de prueba; Hewage *et al.* [14] encuentran que una red convolucional temporal supera a la LSTM con datos de una estación local, y Hou *et al.* [15] combinan capas convolucionales y LSTM para el pronóstico horario. La guía de TensorFlow para series de tiempo [16] describe la construcción de ventanas de entrada y salida que se usa aquí. Un punto común en esta literatura es que la ganancia de un modelo debe medirse frente a referencias ingenuas bien escogidas, como la persistencia o la repetición del ciclo estacional [17], porque en series con un ciclo diario marcado esas reglas ya son difíciles de superar.

B. Fashion-MNIST y aprendizaje por transferencia

Fashion-MNIST [6] conserva el formato de MNIST — 70,000 imágenes de 28×28 píxeles en escala de grises y diez clases— pero es bastante más difícil. En el tablero de resultados de sus autores [18] una persona acierta el 83.5 %, una red de dos capas convolucionales ronda el 91.6 %, VGG16 con 26 millones de parámetros llega al 93.5 %, MobileNet al 95.0 % y una red residual ancha al 96.7 %. Bhatnagar *et al.* [19] reportan 92.54 % con una CNN de dos capas con normalización por lotes y conexiones residuales, y Nocentini *et al.* [20] alcanzan 94.04 % con una red de quince capas y recopilan valores entre 92.3 % y 94 % para variantes de VGG, ResNet18 y MobileNet. El aprendizaje por transferencia reutiliza las características aprendidas en ImageNet [8]: las capas iniciales son genéricas y las finales, específicas de la tarea original [21], y los modelos más exactos en ImageNet tienden a transferir mejor, aunque la ventaja se reduce cuando el dominio de destino es muy distinto [22].

III. PUNTO 1: PRONÓSTICO DE TEMPERATURA CON REDES RECURRENTES

A. Datos y selección de la estación

MeteoNet publica, entre otros productos, las observaciones de las estaciones terrestres del noroeste y del sureste de Francia entre 2016 y 2018, con registros cada 6 minutos de temperatura, punto de rocío, humedad, presión a nivel del mar, viento y precipitación [5]. Se descargaron los tres años de la zona noroeste (287 estaciones) y se resumió la calidad de cada una: cobertura frente a los 262,800 registros esperados, porcentaje de faltantes por variable y presencia en los tres años. Se exigió una cobertura superior al 95 %, menos del 2 % de faltantes en temperatura y menos del 5 % en presión, porque muchas estaciones no reportan esta variable. Cumplieron 46 estaciones y se escogió la 62548002, en la costa cerca de Calais (50.96° N, 1.96° E, 3 m sobre el nivel del mar), por ser la de mejor calidad: 99.8 % de cobertura, ningún dato de temperatura faltante en los registros de 6 minutos y las siete variables disponibles.

Los datos se llevaron a frecuencia horaria (26,304 horas): promedio para las variables de estado, suma para la precipitación y promedio vectorial del viento, que se representa con sus componentes zonal u y meridional v para evitar el salto

de 359° a 0° en la dirección. Tras el remuestreo quedaron 24 horas sin temperatura (0.09 %) repartidas en seis intervalos de datos faltantes. Los cinco de hasta 6 h se interpolaron linealmente y el de 9 h, del 21 y 22 de enero de 2018, se dejó vacío, de modo que las ventanas que lo contienen se descartan: interpolar casi una noche completa sería inventar el ciclo diario. Las 15 horas imputadas se marcaron y se excluyen de las métricas.

B. Análisis exploratorio

La Fig. 1 resume la estructura de la serie. El ciclo anual domina —la mediana mensual va de unos 5°C en febrero a 18°C en julio— y el ciclo diario tiene una amplitud moderada, de 2.8°C en invierno a 5.9°C en verano, propia de una estación costera en la que el mar amortigua los extremos. La autocorrelación (Fig. 2a) conserva picos altos en los múltiplos de 24 h: 0.90 a un día, 0.81 a tres y 0.74 a siete, lo que indica que varios días de historia contienen información útil. El error de la persistencia (Fig. 2b) muestra que la dificultad no crece de forma monótona: es de 0.56°C a 1 h, sube a 3.36°C a 12 h y vuelve a bajar a 2.08°C a 24 h, porque a un día de distancia se compara la misma hora.

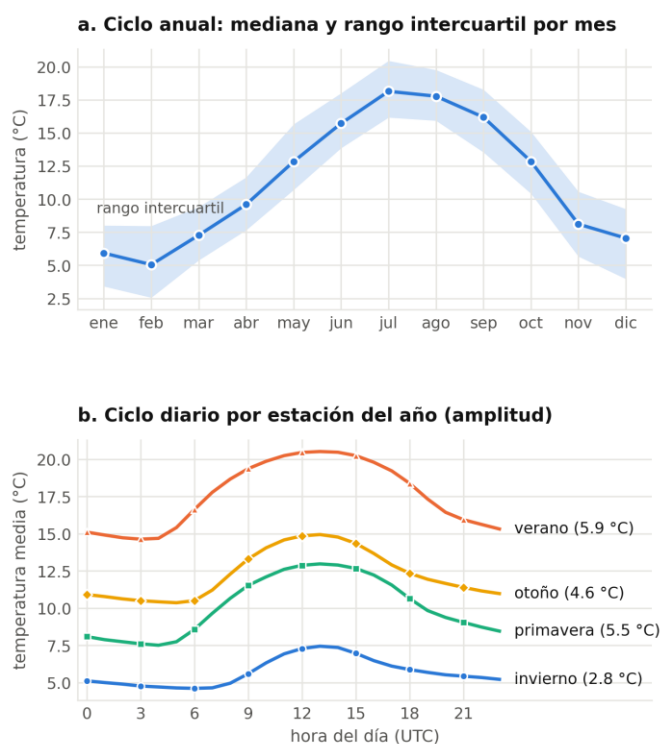


Fig. 1. Estructura de la temperatura en la estación 62548002 (2016–2018). (a) Mediana mensual y rango intercuartil: el ciclo anual abarca unos 13°C . (b) Temperatura media por hora del día según la estación del año; entre paréntesis, la amplitud del ciclo diario.

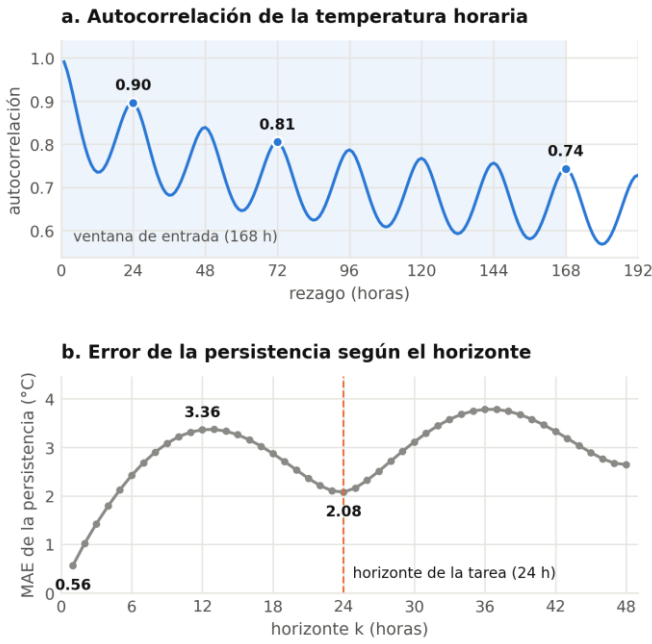


Fig. 2. (a) Autocorrelación de la temperatura horaria; la zona sombreada es la ventana de entrada elegida (168 h). (b) Error absoluto medio de la persistencia según el horizonte, calculado sobre toda la serie.

C. Formulación de la tarea

Se formuló un pronóstico multisalida: a partir de las últimas $L = 168$ horas de $F = 12$ variables, la red produce en una sola pasada las temperaturas de las $H = 24$ horas siguientes:

$$\hat{y}_{t+1:t+H} = f_{\theta}(x_{t-L+1:t}), \quad x_t \in \mathbb{R}^F \quad (1)$$

donde x_t contiene la temperatura, el punto de rocío, la humedad relativa, la presión a nivel del mar, la velocidad del viento y sus componentes u y v , la precipitación y cuatro variables de calendario: el seno y el coseno de la hora del día y del día del año, que le indican a la red la fase de los dos ciclos sin la discontinuidad entre las 23 y las 0 horas. La pérdida de entrenamiento es el error cuadrático medio sobre la salida estandarizada, y las métricas de evaluación son el error absoluto medio (MAE) y la raíz del error cuadrático medio (RMSE) en grados Celsius, promediados sobre las N muestras y los H horizontes:

$$\text{MAE} = \frac{1}{NH} \sum_{i=1}^N \sum_{h=1}^H |\hat{y}_{i,h} - y_{i,h}| \quad (2)$$

El horizonte de 24 h se eligió porque corresponde al uso habitual —el pronóstico del día siguiente— y porque, como muestra la Fig. 2b, no es trivial. La propuesta inicial de ventana era de 72 h, sustentada en la autocorrelación; la longitud final de 168 h salió de la búsqueda de la Sección III-F. Como referencia se entrenó también la misma arquitectura para un horizonte de 1 h.

D. Partición y prevención de la fuga de información

La serie se partió por fechas y sin barajar (Fig. 3): el 59.5 % para entrenamiento (enero de 2016 a octubre de 2017), el 10.5 % para validación (octubre de 2017 a febrero de 2018) y el 30 % final para prueba (febrero a diciembre de 2018). Para

evitar fugas: 1) la media y la desviación estándar de la estandarización se calcularon solo con entrenamiento; 2) las ventanas se construyeron dentro de cada conjunto, de modo que ninguna muestra mezcla horas de dos conjuntos; 3) la climatología usada como línea base se calculó solo con entrenamiento, y 4) el conjunto de prueba no intervino en ninguna decisión. Con $L = 168$ h resultan 15,459 ventanas de entrenamiento, 2,371 de validación y 7,701 de prueba. Una limitación de esta partición es que la prueba cubre primavera, verano y otoño, pero solo unas tres semanas de invierno.

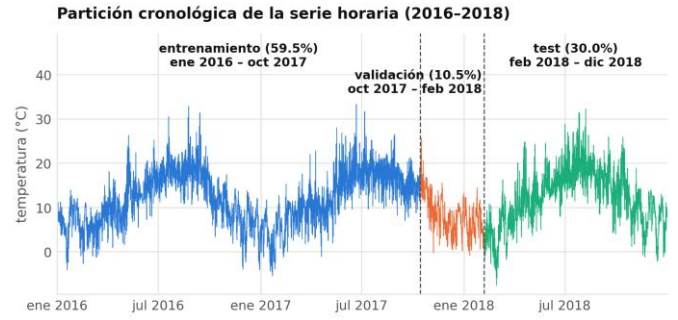


Fig. 3. Partición cronológica de la serie horaria. Las líneas punteadas marcan los cortes; el conjunto de prueba es el 30 % final.

E. Líneas base, arquitectura y protocolo de búsqueda

Se definieron tres líneas base sin aprendizaje, evaluadas sobre las mismas ventanas de prueba: *persistencia*, en la que las 24 h siguientes repiten la última observación; *ayer*, en la que cada hora repite la misma hora del día anterior, una versión de la referencia estacional ingenua [17], y *climatología*, que asigna la media de entrenamiento para ese mes y esa hora.

La arquitectura consiste en una o dos capas recurrentes (SimpleRNN, LSTM o GRU) que leen la ventana completa, seguidas de una capa densa con 24 salidas. Todas las corridas comparten el optimizador Adam [23] con tasa de aprendizaje inicial de 10^{-3} , lotes de 128, un máximo de 60 épocas, la semilla 42 y dos *callbacks* que reaccionan al estancamiento de la optimización: EarlyStopping sobre la pérdida de validación, con paciencia de 8 épocas y restauración de los mejores pesos [24], y ReduceLROnPlateau, que reduce la tasa a la mitad tras 4 épocas sin mejora. Se usó *dropout* [25] a la salida de las capas recurrentes y no *dropout* recurrente, porque este último desactiva la implementación cuDNN y hace el entrenamiento entre cinco y diez veces más lento.

La búsqueda se hizo por etapas (Tabla I), variando una dimensión a la vez a partir de la mejor configuración de la etapa anterior y decidiendo siempre por el MAE de validación. Al final se añadió una verificación que no participa en la selección: repetir la comparación de celdas con la configuración final. En total fueron 18 corridas y unos 11 minutos en una GPU T4 de Google Colab con TensorFlow 2.20 [26].

TABLA I
ETAPAS DE LA BÚSQUEDA DE HIPERPARÁMETROS DEL PUNTO 1

Etapas	Qué se varía	Valores probados
1	Tipo de celda	SimpleRNN, LSTM, GRU
2	Unidades × capas	{32, 64, 128} × {1, 2}
3	Dropout × regularización L2	{0, 0.2, 0.4} × {0, 10 ⁻⁴ }
4	Ventana de entrada	{24, 48, 72, 168} h

F. Resultados de la búsqueda

La Fig. 4 y la Tabla II resumen las 18 corridas. En la etapa 1, con 64 unidades y 72 h, SimpleRNN (1.523 °C) y GRU (1.525 °C) empataron y LSTM quedó atrás (1.615 °C). Las curvas de entrenamiento explican la diferencia: LSTM y GRU alcanzaron su mejor época en la 5 y en la 6, y desde ahí el error de validación subió mientras el de entrenamiento seguía bajando; la SimpleRNN, en cambio, mejoró hasta la época 14. Con 2.5 a 3.3 veces más parámetros para las mismas 15,555 muestras, las celdas con compuertas sobreajustan antes.

TABLA II
BÚSQUEDA DE HIPERPARÁMETROS DEL PUNTO 1 (MAE EN °C)

Etapas	Configuración	MAE ent.	MAE val.	Mejor época / total	Parám.
1	SimpleRNN, 64 u, 72 h	1.266	1.523	14 / 22	6,488
	GRU, 64 u, 72 h	1.293	1.525	6 / 14	16,536
	LSTM, 64 u, 72 h	1.258	1.615	5 / 13	21,272
2	32 u × 1 capa	1.298	1.454	24 / 32	2,232
	32 u × 2 capas	1.222	1.533	31 / 39	4,312
	64 u × 2 capas	1.314	1.543	7 / 15	14,744
	128 u × 2 capas	1.107	1.602	9 / 17	54,040
	128 u × 1 capa	1.395	1.618	3 / 11	21,144
3	dropout 0, L2 10 ⁻⁴	1.302	1.455	23 / 31	2,232
	dropout 0.2, L2 0	1.358	1.500	35 / 43	2,232
	dropout 0.2, L2 10 ⁻⁴	1.357	1.501	35 / 43	2,232
	dropout 0.4, L2 10 ⁻⁴	1.401	1.521	51 / 59	2,232
	dropout 0.4, L2 0	1.400	1.521	51 / 59	2,232
4	ventana 168 h	1.260	1.436	41 / 49	2,232
	ventana 48 h	1.228	1.440	47 / 55	2,232
	ventana 24 h	1.259	1.481	37 / 45	2,232
V	GRU, 32 u, 168 h	1.272	1.500	17 / 25	5,208
	LSTM, 32 u, 168 h	1.230	1.639	10 / 18	6,552

Etapas V: verificación con la configuración final, que no participa en la selección. Las corridas que se repiten entre etapas aparecen una sola vez.

En la etapa 2 ganó la red más pequeña: 32 unidades y una capa (1.454 °C). Apilar dos capas no ayudó en ningún tamaño, y la red de 128 unidades y dos capas es el caso extremo de sobreajuste: tiene el menor error de entrenamiento de toda la búsqueda (1.107 °C) y uno de los mayores de validación (1.602 °C). En la etapa 3 la regularización no aportó: L2 = 10⁻⁴ dio prácticamente lo mismo que no regularizar (1.455 frente a 1.454 °C), y el *dropout* empeoró el error (1.500 y 1.521 °C con 0.2 y 0.4) y alargó el entrenamiento. Es coherente con una brecha de apenas 0.15 °C entre entrenamiento y validación: no había sobreajuste que corregir. En la etapa 4, las ventanas de 48 h (1.440 °C) y 168 h (1.436 °C) quedaron prácticamente empatadas, la de 72 h un poco atrás (1.455 °C) y la de 24 h claramente peor (1.481 °C): con un solo ciclo diario la red no ve la tendencia de varios días. Por el criterio acordado se eligió 168 h, aunque 48 h habría sido una alternativa casi igual de

buena y de la mitad del costo.

La verificación confirmó la elección de la celda con la configuración final: SimpleRNN 1.436 °C, GRU 1.500 °C y LSTM 1.639 °C, de nuevo con parada temprana en las épocas 17 y 10 para las celdas con compuertas, frente a la 41 de la SimpleRNN. El modelo final es una SimpleRNN de 32 unidades y una capa, sin *dropout*, con L2 = 10⁻⁴ y ventana de 168 h, con 2,232 parámetros entrenables. Cada configuración se entrenó con una sola semilla, así que las diferencias de milésimas de grado no son significativas y la ejecución en GPU no es del todo determinista.

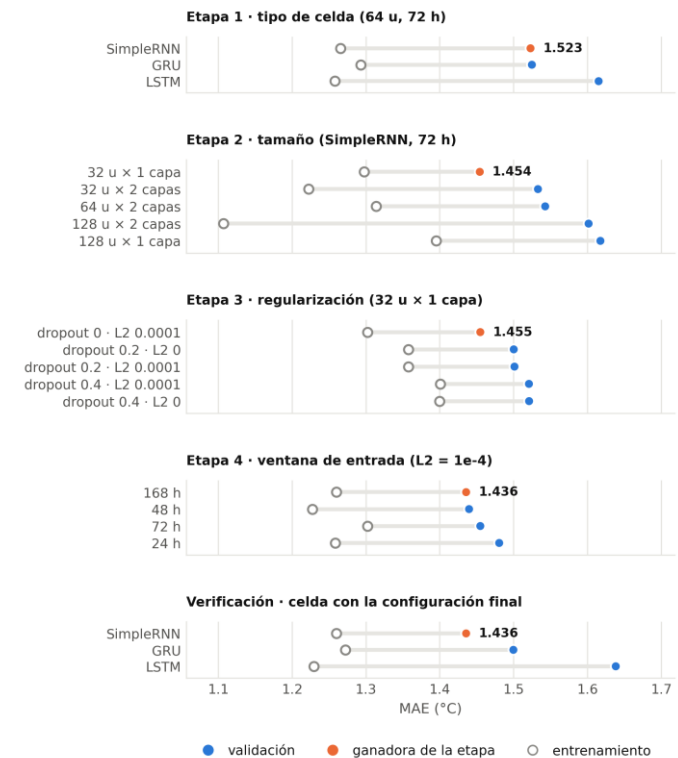


Fig. 4. Búsqueda de hiperparámetros del Punto 1. Cada fila es una corrida: el punto relleno es el MAE de validación y el punto vacío, el de entrenamiento; la distancia entre ambos mide el sobreajuste. En naranja, la ganadora de cada etapa.

G. Evaluación en el conjunto de prueba

La Tabla III compara el modelo final con las líneas base sobre las 7,701 ventanas de prueba. La RNN obtiene un MAE de 1.602 °C y un RMSE de 2.142 °C, un 24.7 % por debajo de «ayer», la línea base más fuerte (2.128 °C); la persistencia y la climatología quedan en 2.785 y 2.871 °C. La Fig. 5 muestra que la ventaja se mantiene en los 24 horizontes: el error de la RNN crece de forma suave, de 0.54 °C a 1 h a 1.94 °C a 24 h. La mayor ganancia está entre 4 y 20 h, donde la persistencia se dispara; a 24 h la ventaja sobre «ayer» se reduce (1.94 frente a 2.13 °C), porque en ese punto el ciclo diario ya le da a esa regla casi toda la información. Con el horizonte de referencia de 1 h, la misma arquitectura logra 0.41 °C frente a 0.61 °C de la persistencia.

TABLA III
RESULTADOS DEL PUNTO 1 EN EL CONJUNTO DE PRUEBA

Modelo	MAE (°C)	RMSE (°C)	Cambio frente a «ayer»
SimpleRNN (modelo final)	1.602	2.142	+24.7 %
«Ayer» (misma hora del día anterior)	2.128	2.804	—
Persistencia	2.785	3.805	−30.8 %
Climatología (mes × hora)	2.871	3.691	−34.9 %

Promedios sobre 7,701 ventanas y 24 horizontes, sin las horas imputadas.

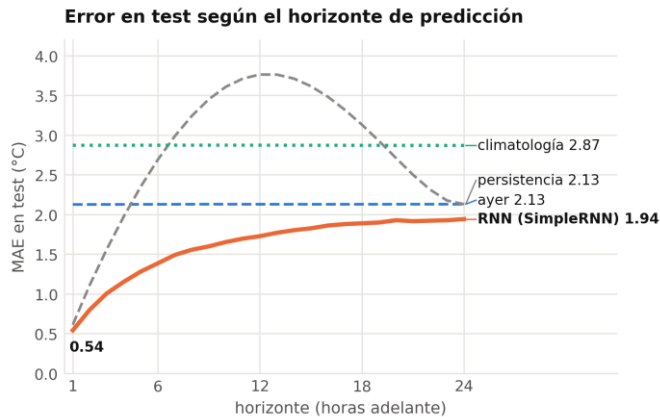


Fig. 5. MAE en el conjunto de prueba según el horizonte. La RNN queda por debajo de las tres líneas base en los 24 horizontes; a la derecha, los valores a 24 h.

El análisis por mes y por hora de emisión (Fig. 6) muestra que la RNN le gana a «ayer» en todos los meses y que la ventaja es mayor en los meses de transición, cuando el tiempo cambia bruscamente de un día a otro: en abril el error baja de 2.71 a 1.78 °C. Los meses más fáciles fueron mayo y junio (1.41 y 1.47 °C); en junio ambos modelos casi empatan porque los días se parecen mucho entre sí. Por hora de emisión, los pronósticos hechos de madrugada (3 a 5 UTC) son los peores (1.72 °C) y los emitidos hacia el mediodía, los mejores (1.50 °C): a mediodía la red ya observó cómo empezó el día, mientras que de madrugada debe anticipar el máximo de la tarde con la información del día anterior.

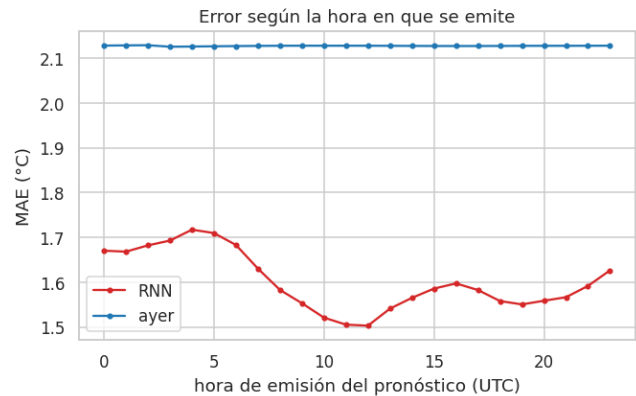
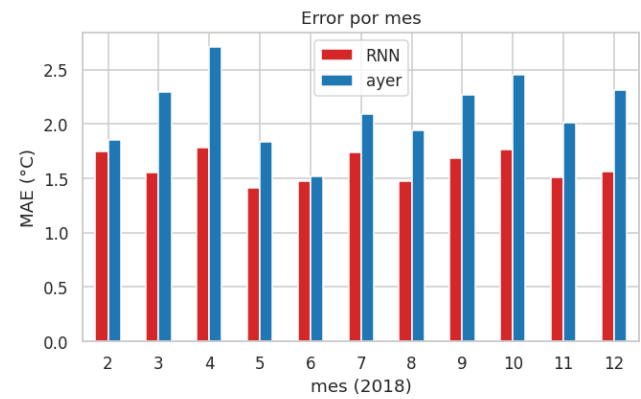


Fig. 6. MAE de la RNN y de la línea base «ayer» en el conjunto de prueba, (arriba) por mes de 2018 y (abajo) según la hora en que se emite el pronóstico.

Los ejemplos de la Fig. 7 ilustran tres situaciones. En el mejor día (17 de mayo, MAE de 0.20 °C) el ciclo es regular y la red lo reproduce casi exactamente. En el día típico (21 de julio, 1.51 °C) la forma es correcta, pero la red suaviza el máximo de la tarde: predice cerca de 21 °C cuando se alcanzaron 23.7 °C. En el peor día (17 de marzo, 6.94 °C) entra un frente frío y la temperatura cae de 6 °C a −1 °C; ni la red ni «ayer» lo anticipan, porque nada en las 168 horas previas de esta estación lo anuncia. El sesgo medio es de +0.11 °C, los errores son simétricos y el 95 % de los errores absolutos está por debajo de 4.4 °C; las colas corresponden a cambios de masa de aire como el del ejemplo.

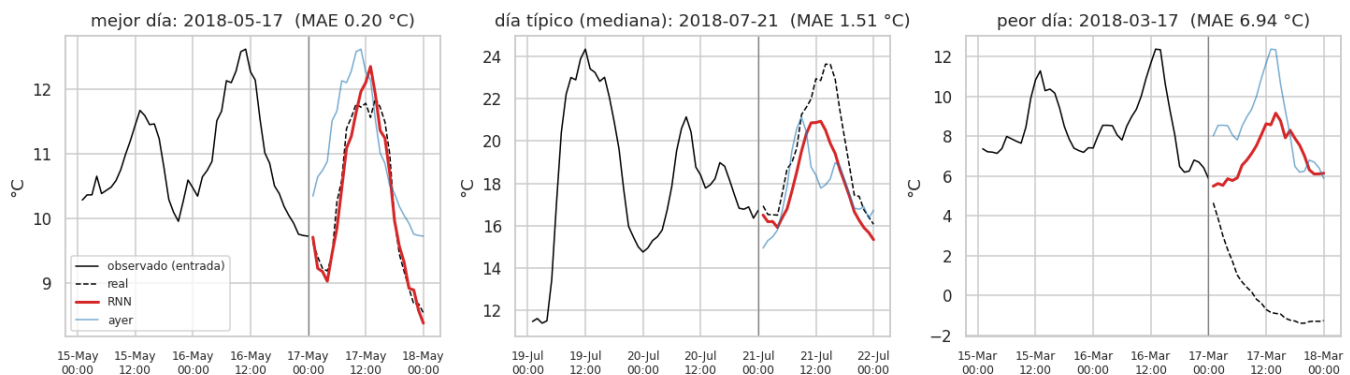


Fig. 7. Pronósticos emitidos a medianoche para el mejor día, un día típico (mediana del error) y el peor día del conjunto de prueba: 48 h observadas, las 24 h reales y las predicciones de la RNN y de «ayer».

H. Discusión

Las cifras de otros trabajos no son directamente comparables, porque dependen del clima de cada sitio, del horizonte y de las variables de entrada. Como orden de magnitud, El-Shawa *et al.* [27] reportan un MAE medio de entre 1.93 y 2.55 °C para horizontes de 1 a 48 h con redes de grafos sobre datos de análisis en el suroeste de Ontario, del mismo orden que el obtenido aquí para 1 a 24 h con una sola estación. Más informativa que esa comparación es la mejora del 24.7 % frente a una regla ingenua fuerte, evaluada con las mismas ventanas.

Lo que funcionó fue la ingeniería de entradas —las variables de calendario le entregan a la red la fase de los ciclos—, una red pequeña y al menos dos días de historia. Lo que no funcionó fue agregar unidades o capas, el *dropout* y las celdas con compuertas. Que la SimpleRNN supere a LSTM y GRU no contradice la literatura [12]: con 168 pasos el desvanecimiento del gradiente [11] sigue presente, pero la información más útil está en las últimas horas y en las variables de calendario, de modo que la memoria de largo plazo de las compuertas no compensa la capacidad adicional que traen. El límite del modelo está en la información: los peores días son frentes que una sola estación no puede anticipar sin datos espaciales o de un modelo numérico.

IV. PUNTO 2: CLASIFICACIÓN DE FASHION-MNIST CON REDES CONVOLUCIONALES

A. Datos y diseño experimental

Fashion-MNIST tiene 60,000 imágenes de desarrollo y 10,000 de prueba, de 28×28 píxeles en escala de grises y diez clases balanceadas (Fig. 8). Antes de partir los datos se verificó su integridad: tipo uint8, rango de 0 a 255, diez etiquetas, ninguna imagen repetida dentro del desarrollo y ninguna imagen de prueba repetida en el desarrollo. El desarrollo se partió de forma estratificada en 54,000 imágenes de entrenamiento y 6,000 de validación (5,400 y 600 por clase); la prueba conserva sus 1,000 imágenes por clase y solo se usó en la evaluación final.

Fashion-MNIST: una muestra por clase

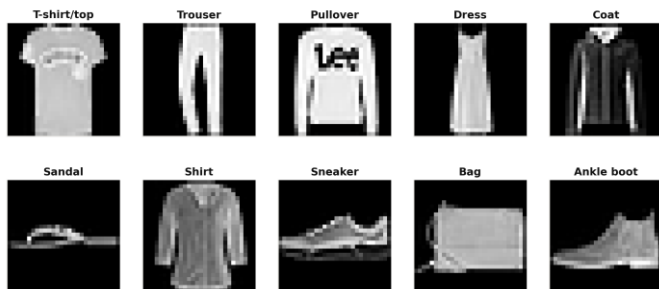


Fig. 8. Una imagen de cada clase de Fashion-MNIST (28×28 píxeles).

Para que la comparación fuera justa, las tres aproximaciones comparten la partición, la semilla 42, el mismo flujo de datos con lotes de 128, la pérdida de entropía cruzada, el optimizador Adam, los mismos *callbacks* —EarlyStopping con paciencia de 3 épocas sobre la pérdida de validación y ReduceLROnPlateau

con factor 0.3 y paciencia 2— y las mismas métricas. Todas reciben la imagen original de $28 \times 28 \times 1$: la normalización, el redimensionamiento a 96×96 , la conversión a tres canales y el preprocesamiento de ImageNet ocurren dentro de cada modelo, de modo que no conviven dos versiones de los datos. Se comprobó numéricamente que ese preprocesamiento coincide con las funciones oficiales de Keras.

B. Arquitecturas

La Tabla IV resume las tres redes. La *CNN desde cero* tiene dos bloques de convolución 3×3 con 32 y 64 filtros, normalización por lotes [28], ReLU y *max pooling*, seguidos de agrupamiento promedio global (GAP), una capa densa de 128 unidades, *dropout* y la capa *softmax*. Trabaja en la resolución nativa y tiene 28,714 parámetros. El diseño es compacto a propósito: imágenes de 28×28 con fondo uniforme no necesitan una jerarquía profunda, y el agrupamiento global evita la gran capa densa de las arquitecturas clásicas.

VGG16 con filtros de ImageNet reutiliza las trece capas convolucionales de VGG16 [7] con los pesos de ImageNet y reemplaza la parte clasificadora por GAP, *dropout* y *softmax*. La imagen se redimensiona a 96×96 y el gris se replica en los tres canales, centrado con las medias de ImageNet. *MobileNetV2* se usa con multiplicador de ancho 1.0 y pesos de ImageNet para entradas de 96×96 [9]; reemplaza las convoluciones estándar por bloques residuales invertidos con convoluciones separables en profundidad, la entrada se escala a $[-1, 1]$ y la cabeza es igual a la de VGG16. La resolución de 96×96 equilibra el costo con un tamaño razonable para los filtros de ImageNet; Keras exige al menos 32×32 y, a esa resolución, los mapas de la última capa serían de 1×1 .

Las dos redes preentrenadas siguen el esquema de dos fases recomendado para la transferencia [29]: primero se entrena solo la cabeza con la red base (*backbone*) congelada y luego se hace ajuste fino (*fine-tuning*) de las últimas capas del *backbone* —el último bloque convolucional de VGG16 y los dos últimos bloques residuales de MobileNetV2 con su convolución final— con una tasa de aprendizaje diez veces menor y con las capas de normalización por lotes congeladas.

TABLA IV
ARQUITECTURAS DEL PUNTO 2

Modelo	Arquitectura	Parámetros	Entrenables
CNN desde cero	Conv 3×3 (32) y Conv 3×3 (64), cada una con BN, ReLU y MaxPool; GAP, Dense 128, dropout y softmax 10. Entrada $28 \times 28 \times 1$	28,714	28,522
VGG16	13 capas convolucionales con pesos de ImageNet; GAP, dropout y softmax 10. Entrada $96 \times 96 \times 3$	14,719,818	5,130 7,084,554
MobileNet V2	Bloques residuales invertidos ($\alpha = 1.0$) con pesos de ImageNet; GAP, dropout y softmax 10. Entrada $96 \times 96 \times 3$	2,270,794	12,810 1,207,690

Entrenables de las preentrenadas: con el *backbone* congelado (arriba) y durante el ajuste fino (abajo). BN: normalización por lotes; GAP: agrupamiento promedio global.

C. Búsqueda de hiperparámetros

La búsqueda fue manual y estructurada, con el mismo presupuesto para las tres familias: las 54,000 imágenes de entrenamiento, 12 épocas por corrida y pesos nuevos en cada una. Se variaron la capacidad (solo en la CNN: 32–64 frente a 64–128 filtros), el *dropout*, la aumentación de datos —traslaciones de hasta el 8 % y rotaciones de hasta $\pm 21.6^\circ$ — y la tasa de aprendizaje (10^{-3} frente a 10^{-4}). La selección usó la mayor exactitud de validación y, como desempate, la menor pérdida. La Tabla V y la Fig. 9 muestran las diez corridas.

En las tres familias ganó la misma receta: sin aumentación y con tasa de aprendizaje de 10^{-3} (CNN 87.65 %, VGG16 88.30 % y MobileNetV2 90.13 % en validación). Las corridas con aumentación quedaron entre 3.9 y 5.5 puntos por debajo, aunque hay que leer esa diferencia con cuidado: también tenían más *dropout*, así que el efecto no puede atribuirse solo a la aumentación. La tasa de 10^{-4} quedó subentrenada en las tres familias, con curvas que seguían subiendo al final de las 12 épocas. Duplicar los filtros de la CNN (64–128) apenas cambió la validación (82.53 % frente a 82.12 %, ambas con aumentación) y triplicó los parámetros. El panel derecho de la Fig. 9 aclara un detalle: la brecha entre entrenamiento y validación es negativa en casi todas las corridas porque la exactitud de entrenamiento se mide con *dropout* y aumentación activos. No es un error; indica que ninguna configuración sobreajustaba en 12 épocas, de modo que la aumentación, útil cuando hay sobreajuste, aquí solo hizo la tarea más difícil dentro de un presupuesto corto.

TABLA V
BÚSQUEDA DE HIPERPARÁMETROS DEL PUNTO 2

Modelo	Configuración	Mejor época	Exact. val. (%)	Pérd. val.	Tiempo (s)
CNN desde cero	32–64, dropout 0, sin aum., lr 10^{-3}	11	87.65	0.351	40
	64–128, dropout 0.3, con aum., lr 10^{-3}	10	82.53	0.492	91
	32–64, dropout 0.3, con aum., lr 10^{-3}	12	82.12	0.494	72
	32–64, dropout 0.3, con aum., lr 10^{-4}	12	72.53	0.751	60
VGG16	dropout 0.2, sin aum., lr 10^{-3}	10	88.30	0.324	889
	dropout 0.4, con aum., lr 10^{-3}	12	84.37	0.437	621
	dropout 0.4, con aum., lr 10^{-4}	12	82.98	0.519	617
MobileNet V2	dropout 0.2, sin aum., lr 10^{-3}	10	90.13	0.267	150
	dropout 0.3, con aum., lr 10^{-3}	10	86.17	0.373	238
	dropout 0.3, con aum., lr 10^{-4}	12	84.18	0.418	236

Doce épocas por corrida sobre las 54,000 imágenes de entrenamiento; aum.: aumentación de datos.

D. Entrenamiento final y ajuste fino

Con la mejor configuración de cada familia se entrenó sobre las 54,000 imágenes hasta 30 épocas, más 15 de ajuste fino para las preentrenadas, siempre con parada temprana (Fig. 10). La CNN usó las 30 épocas y su mejor época fue la 28, con 89.47 % en validación. VGG16 se estancó en la fase congelada (88.67 % en la época 26), por debajo de la CNN propia, y el ajuste fino le dio el mayor salto del experimento: hasta 93.65 % en validación. MobileNetV2 partió más alto con el *backbone* congelado (90.77 %) y el ajuste fino la llevó a 93.17 %. El contraste confirma que las características de ImageNet no se transfieren tal cual a prendas pequeñas en escala de grises [21], [22]: el cambio de dominio exige adaptar al menos el último bloque. Hay que declarar también que la CNN no tuvo fase de ajuste fino, así que las preentrenadas recibieron más épocas de entrenamiento en total.

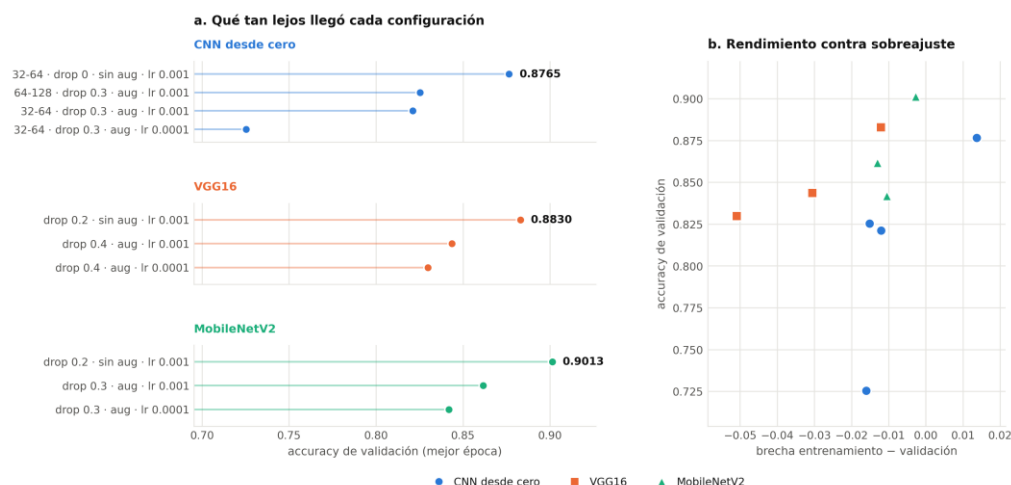


Fig. 9. Búsqueda de hiperparámetros del Punto 2. (a) Exactitud de validación de cada configuración, agrupada por familia; se rotula la ganadora. (b) Exactitud de validación frente a la brecha entre entrenamiento y validación.

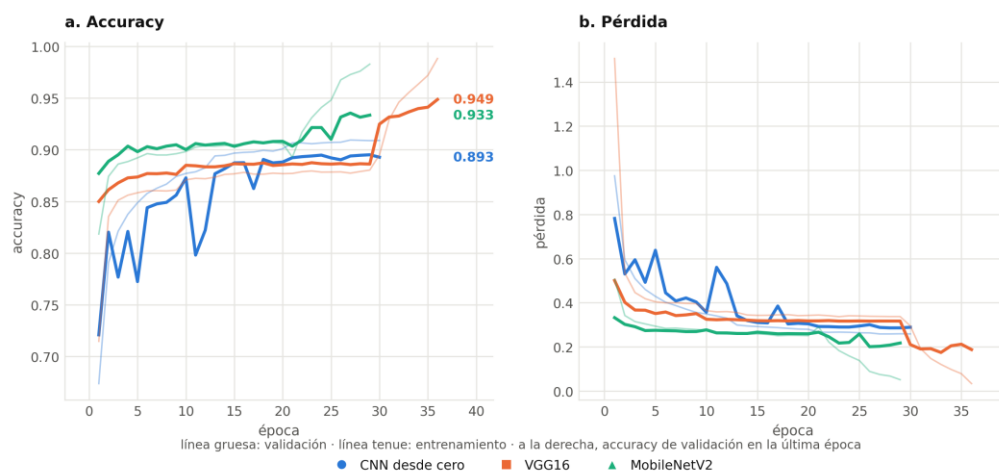


Fig. 10. Curvas del entrenamiento final. La línea gruesa es validación y la tenue, entrenamiento; el salto de VGG16 y MobileNetV2 cerca de la época 30 y la 20 corresponde al inicio del ajuste fino. Los valores de la derecha son la exactitud de validación en la última época; los pesos restaurados son los de la menor pérdida de validación.

E. Evaluación en el conjunto de prueba

La Tabla VI resume la evaluación única sobre las 10,000 imágenes de prueba. VGG16 obtiene una exactitud de 93.19 %, MobileNetV2 de 92.43 % y la CNN desde cero de 89.04 %, con el mismo orden en F1 macro y en *log-loss*. Los intervalos de confianza del 95 %, calculados con 1,000 remuestreos *bootstrap* del conjunto de prueba [30], separan con claridad a la CNN de las dos preentrenadas, pero los de VGG16 [92.69, 93.67] y MobileNetV2 [91.90, 92.95] se solapan. Esa lectura es conservadora, porque ignora que las dos redes se evalúan sobre las mismas imágenes: la comparación pareada de la Sección IV-F muestra que la diferencia de 0.76 puntos sí es significativa, aunque pequeña. La Fig. 11 pone esa diferencia en contexto: VGG16 tiene 6.5 veces más parámetros, casi seis veces más parámetros entrenables durante el ajuste fino y usó ocho veces más tiempo de GPU (2,687 s frente a 340 s).

El análisis por clase (Fig. 12 y Fig. 13) muestra que las tres redes coinciden en qué es difícil. Pantalón, bolso, sandalia,

botín y tenis superan 0.94 de F1 en todos los modelos, mientras que camisa (*Shirt*) es la clase más difícil: F1 de 0.71 en la CNN, 0.78 en MobileNetV2 y 0.79 en VGG16. Los pares más confundidos se repiten en los tres modelos: camisa con camiseta (entre 95 y 131 casos) y camiseta con camisa, seguidos de las confusiones entre buzo, abrigo y camisa. Son prendas de silueta parecida que a 28×28 píxeles se distinguen por detalles como el cuello o los botones. La ventaja de las preentrenadas se concentra justamente en las prendas superiores —camiseta, buzo, vestido, abrigo y camisa—, donde VGG16 mejora el F1 de la CNN entre 5 y 9 puntos.

La Fig. 14 muestra los quince errores más confiados de VGG16, todos con probabilidad cercana al 100 %. Varios parecen errores de etiquetado del conjunto de datos más que del modelo: dos imágenes rotuladas como pantalón y una como camiseta tienen forma de vestido, y una rotulada como camisa parece un bolso. Este fenómeno está documentado en otros conjuntos de referencia [32] e implica que una parte del error residual no se puede reducir con una mejor arquitectura.

TABLA VI
COMPARACIÓN DE LAS TRES APROXIMACIONES EN EL CONJUNTO DE PRUEBA (10,000 IMÁGENES)

Modelo	Exactitud (%)	IC 95 % (%)	F1 macro	Precisión macro	Exhaust. macro	Log-loss	Parámetros	Entrenables	Tiempo (s)
VGG16	93.19	[92.69, 93.67]	0.9319	0.9327	0.9319	0.1925	14,719,818	7,084,554	2,687
MobileNetV2	92.43	[91.90, 92.95]	0.9244	0.9248	0.9243	0.2301	2,270,794	1,207,690	340
CNN desde cero	89.04	[88.46, 89.63]	0.8904	0.8906	0.8904	0.3095	28,714	28,522	79

IC 95 %: intervalo de confianza de la exactitud por bootstrap (1,000 remuestreos). Entrenables: en la fase final. Tiempo: entrenamiento total en una GPU T4, incluido el ajuste fino.

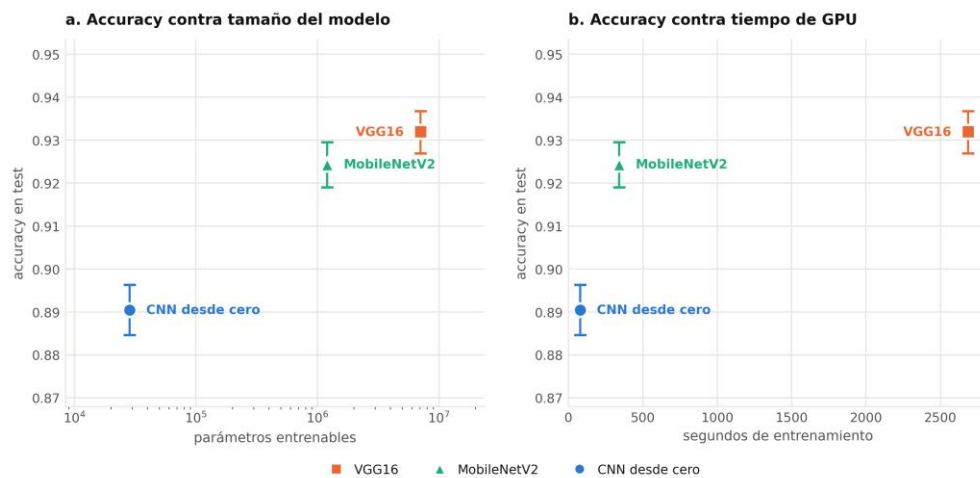


Fig. 11. Exactitud en prueba con su intervalo de confianza del 95 % frente a (a) los parámetros entrenables, en escala logarítmica, y (b) el tiempo de entrenamiento en GPU.

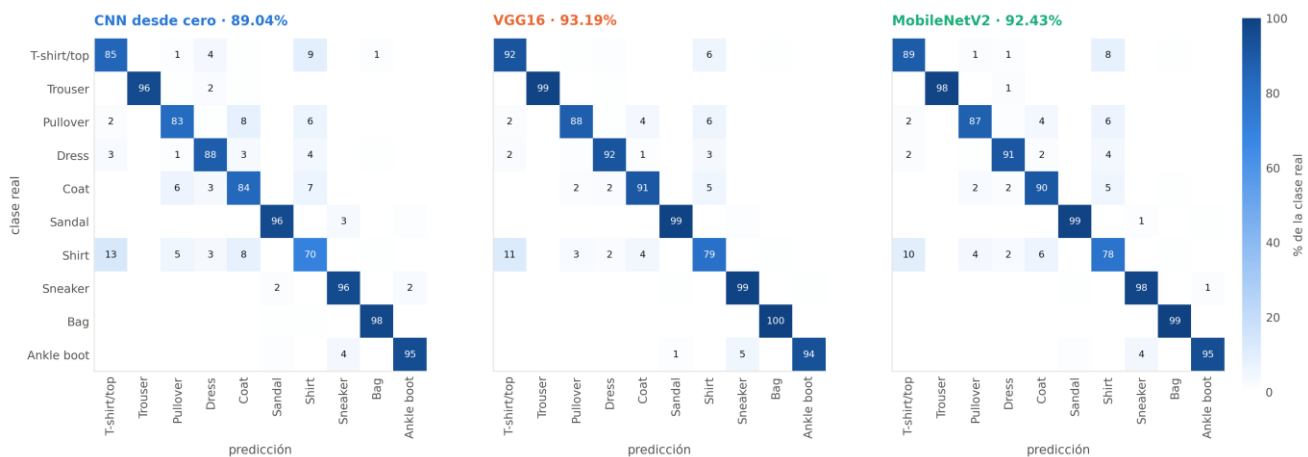


Fig. 12. Matrices de confusión normalizadas por la clase real (%). La diagonal es la exhaustividad de cada clase; la confusión dominante en los tres modelos es camisa (Shirt) con camiseta (T-shirt/top).

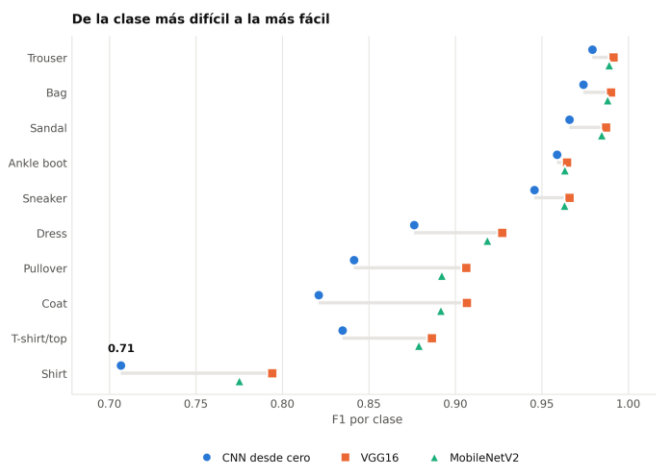


Fig. 13. F1 por clase de los tres modelos, de la clase más difícil a la más fácil. La línea gris une el peor y el mejor modelo de cada clase.

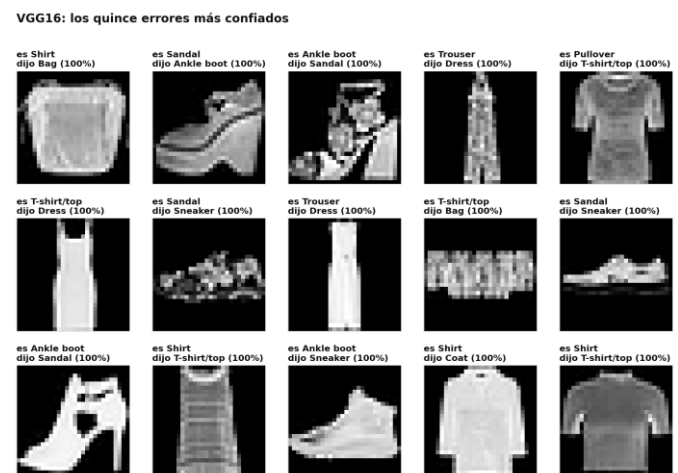


Fig. 14. Los quince errores más confiados de VGG16 en prueba. «es» indica la etiqueta del conjunto de datos y «dijo», la predicción con su probabilidad.

F. Extensión a una red moderna: EfficientNetV2-B0

Como bono se extendió la comparación a EfficientNetV2-B0 [33], la continuación de la línea de MobileNetV2: combina

bloques MBConv y Fused-MBConv, escala de forma conjunta la profundidad, el ancho y la resolución, y tiene 5.9 millones de parámetros. Se repitió exactamente el protocolo de las otras tres redes —misma partición, semilla, lotes, *callbacks* y rejilla de búsqueda, ajuste fino de las últimas 20 capas con la tasa dividida por diez, float32 y la misma GPU T4—. La única diferencia es que Keras incluye el preprocesamiento de ImageNet dentro del modelo, así que la red recibe la imagen en el rango de 0 a 255 replicada en tres canales. La búsqueda repitió el patrón de la Tabla V: ganó dropout 0.2 sin aumentación y tasa de 10^{-3} (90.77 % en validación), la aumentación bajó a 87.48 % y la tasa de 10^{-4} quedó subentrenada (84.80 %).

Con el *backbone* congelado, la exactitud de validación se estancó cerca del 91 % y el ajuste fino la llevó al 93.8 %; las 15 épocas de esa fase corrieron completas sin que se activara la parada temprana, así que con más presupuesto probablemente ganaría algo más. En prueba obtuvo 92.73 % de exactitud (IC 95 % [92.18, 93.24]), F1 macro de 0.9275 y *log-loss* de 0.2158, con 603 s de entrenamiento y 0.82 millones de parámetros entrenables en la fase final. Queda entre VGG16 y MobileNetV2 tanto en exactitud como en costo (Fig. 15).

Como los intervalos por separado son conservadores, las redes se compararon además de forma pareada sobre las mismas 10,000 imágenes: con el intervalo *bootstrap* de la diferencia de exactitud y con la prueba exacta de McNemar [31], que solo cuenta las imágenes en que un modelo acierta y el otro no. La Tabla VII resume el resultado contra VGG16, la mejor red. EfficientNetV2-B0 queda a 0.46 puntos, con un intervalo que

incluye el cero y $p = 0.058$: es la única red que no se distingue de VGG16, y lo logra con 4.5 veces menos tiempo. En cambio, la diferencia de 0.76 puntos entre VGG16 y MobileNetV2, que los intervalos separados no permitían afirmar, sí es significativa ($p = 0.0015$). Entre EfficientNetV2-B0 y MobileNetV2 la diferencia, de 0.30 puntos, no lo es ($p = 0.23$). Por clase, EfficientNetV2-B0 repite el patrón de las otras redes: camisa sigue siendo la clase más difícil (F1 de 0.79) y obtiene el mejor F1 de las cuatro en botón (0.97) y bolso (0.99).

G. Ventajas y desventajas de cada aproximación

La Tabla VIII contrasta las cuatro aproximaciones con los resultados obtenidos. Con un criterio de «mejor» que combine exactitud, incertidumbre y costo, EfficientNetV2-B0 es la opción más equilibrada; MobileNetV2 es la más barata de las preentrenadas, a cambio de 0.76 puntos frente a VGG16; VGG16 solo se justifica si cada décima de exactitud importa más que el cómputo, y la CNN propia es la elección cuando el presupuesto de memoria o de energía es muy bajo.

Frente a la literatura, la CNN propia (89.04 %) queda por debajo de las CNN pequeñas mejor ajustadas, como la de Bhatnagar *et al.* (92.54 %) [19], que usa conexiones residuales y más entrenamiento. VGG16 (93.19 %) coincide con el 93.5 % del tablero de Fashion-MNIST para esa arquitectura [18], y MobileNetV2 (92.43 %) queda por debajo del 95.0 % reportado allí para MobileNet con volteos horizontales y del 93.96 % recopilado por Nocentini *et al.* [20]. Las diferencias son razonables dado el presupuesto de 30 épocas, la resolución de 96×96 y la ausencia de aumentación en los modelos finales.

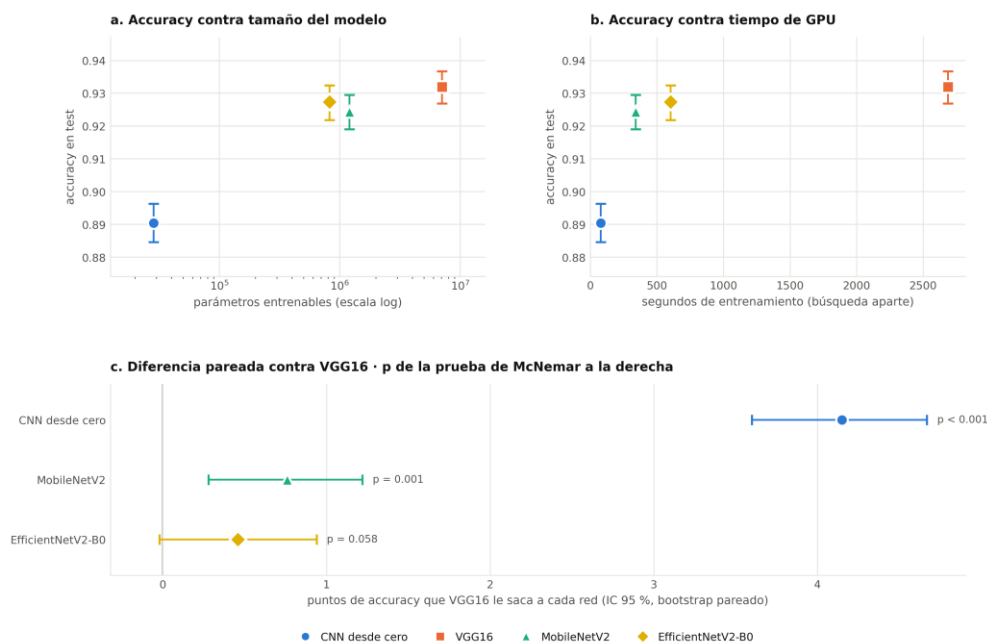


Fig. 15. Extensión a EfficientNetV2-B0. Exactitud en prueba con su intervalo de confianza del 95 % frente a (a) los parámetros entrenables, en escala logarítmica, y (b) el tiempo de entrenamiento en GPU; (c) diferencia pareada de exactitud de VGG16 frente a cada red, con su intervalo *bootstrap* del 95 % y el valor p de la prueba de McNemar. Un intervalo que cruza el cero indica una diferencia no concluyente.

TABLA VII
COMPARACIÓN PAREADA CONTRA VGG16 EN EL CONJUNTO DE PRUEBA

Modelo	Exactitud (%)	Tiempo (s)	Entrenables	Diferencia con VGG16 (puntos)	IC 95 % de la diferencia	Solo acierta VGG16	Solo acierta el modelo	p (McNemar)
VGG16	93.19	2,687	7,084,554	—	—	—	—	—
EfficientNetV2-B0	92.73	603	823,098	0.46	[−0.02, 0.94]	305	259	0.058
MobileNetV2	92.43	340	1,207,690	0.76	[0.28, 1.22]	317	241	0.0015
CNN desde cero	89.04	79	28,522	4.15	[3.60, 4.67]	630	215	< 0.001

Diferencia: exactitud de VGG16 menos la del modelo, con intervalo *bootstrap* pareado (1,000 remuestros). p: prueba exacta de McNemar sobre las imágenes en que solo uno de los dos modelos acierta. Tiempo y entrenables como en la Tabla VI.

TABLA VIII
VENTAJAS Y DESVENTAJAS OBSERVADAS DE CADA APROXIMACIÓN

Aproximación	Ventajas	Desventajas
CNN desde cero	Trabaja en la resolución nativa; 28,714 parámetros y 79 s de entrenamiento; control total del diseño; fácil de desplegar en dispositivos con poca memoria o energía.	No reutiliza conocimiento previo; la menor exactitud (89.04 %) y el peor desempeño en las prendas superiores de silueta parecida; depende más de las decisiones de diseño.
VGG16 (filtros de ImageNet)	La mayor exactitud y el menor log-loss; filtros genéricos reutilizables; el ajuste fino del último bloque aporta cerca de cinco puntos de exactitud en validación.	14.7 millones de parámetros y 45 minutos de GPU; exige redimensionar y replicar canales; congelada rinde menos que la CNN propia; su ventaja sobre MobileNetV2 es de solo 0.76 puntos y sobre EfficientNetV2-B0 no es significativa.
MobileNetV2 (transferencia)	Solo 0.76 puntos por debajo de VGG16 con 6.5 veces menos parámetros y 8 veces menos tiempo; la mejor fase con el <i>backbone</i> congelado; diseñada para dispositivos móviles.	También requiere entrada de 96×96×3; el ajuste fino es sensible a la tasa de aprendizaje y a las capas de normalización; persiste el cambio de dominio respecto a ImageNet.
EfficientNetV2-B0 (bono)	Sin diferencia significativa con VGG16 (92.73 %, p = 0.058) con 4.5 veces menos tiempo y 0.82 millones de parámetros entrenables; trae el preprocesamiento de ImageNet dentro del modelo.	1.8 veces más lenta que MobileNetV2 sin superarla de forma significativa; congelada se estanca en 91 % y depende del ajuste fino; también requiere entrada de 96×96×3.

V. LECCIONES APRENDIDAS

Las líneas base y los intervalos cambian la lectura de los resultados. Sin la regla «ayer», un MAE de 1.60 °C no dice si el modelo es bueno o malo. Y la forma de medir la incertidumbre también importa: con intervalos *bootstrap* separados, VGG16 y MobileNetV2 parecían empatadas, y solo la prueba pareada, que aprovecha que ambas se evalúan sobre las mismas imágenes, mostró que su diferencia de 0.76 puntos es real.

Más capacidad no es mejor por defecto. El modelo ganador del Punto 1 tiene 2,232 parámetros, y en el Punto 2 duplicar los filtros no mejoró la CNN. Con datos limitados o señales regulares, la capacidad sobrante se convierte en sobreajuste que la parada temprana delata en pocas épocas.

La información de entrada pesa tanto como la arquitectura. Codificar la hora y el día con seno y coseno le ahorró a la red descubrir los ciclos, y el límite del pronóstico lo pone la falta de información espacial, no la celda recurrente.

La regularización no es gratuita. El *dropout* en la RNN y la aumentación en las CNN empeoraron la validación porque, dentro del presupuesto, no había sobreajuste que corregir.

La transferencia necesita ajuste fino cuando cambia el dominio. VGG16 congelada rindió menos que la CNN propia; con el último bloque ajustado ganó cinco puntos. EfficientNetV2-B0 repitió el patrón: congelada se estancó cerca del 91 % en validación y el ajuste fino la llevó al 93.8 %.

La reproducibilidad es parte del diseño experimental. Registrar cada corrida, fijar semillas, guardar las predicciones y separar el entrenamiento de la generación de figuras permitió

rehacer tablas y gráficas sin volver a entrenar.

Limitaciones y trabajo futuro. El Punto 1 usa una sola estación, una sola semilla por configuración y una prueba con poco invierno; en el Punto 2 la búsqueda cubrió trece configuraciones de 12 épocas, los modelos finales no se repitieron con varias semillas y el ajuste fino de EfficientNetV2-B0 agotó sus 15 épocas sin estancarse. Como trabajo futuro se propone incorporar estaciones vecinas o los pronósticos del modelo numérico AROME que también publica MeteoNet, probar convoluciones temporales o atención, repetir las corridas con varias semillas, dar más épocas de ajuste fino y extender el bono a ConvNeXt, que el código ya deja preparado.

VI. CONCLUSIONES

En el Punto 1 se construyó un pronosticador de temperatura a 24 h para la estación 62548002 de MeteoNet con una formulación explícita (168 h de entrada, 24 h de salida y 12 variables), una partición cronológica sin fuga de información y una búsqueda sistemática por etapas. El modelo elegido, una SimpleRNN de 32 unidades, logra un MAE de 1.60 °C en el 30 % final de la serie, un 24.7 % mejor que la mejor línea base y mejor en todos los horizontes. Las celdas con compuertas, las redes más grandes y la regularización con *dropout* no aportaron en este problema.

En el Punto 2 se compararon tres aproximaciones en condiciones idénticas y, como extensión, EfficientNetV2-B0. El aprendizaje por transferencia supera con claridad a la CNN desde cero (93.19 %, 92.73 % y 92.43 % frente a 89.04 %). La prueba pareada muestra que VGG16 supera a MobileNetV2 por 0.76 puntos, una diferencia real pero pequeña frente a un costo

ocho veces mayor, y que EfficientNetV2-B0 no se distingue de VGG16 con 4.5 veces menos tiempo. Por eso EfficientNetV2-B0 es la opción recomendada cuando importa la exactitud y MobileNetV2 cuando importa el costo; la CNN propia sigue siendo valiosa cuando los recursos son muy limitados.

En los dos puntos la conclusión general es la misma: un modelo debe juzgarse por cuánto mejora frente a una referencia razonable, con qué incertidumbre y a qué costo, y no solo por el valor aislado de su métrica.

DISPONIBILIDAD DEL CÓDIGO

El código (notebooks de descarga, entrenamiento y figuras), los resultados intermedios, las tablas y las figuras de este informe están en el repositorio público del trabajo [10]. Los notebooks se ejecutaron en Google Colab con una GPU T4 y TensorFlow 2.20; los notebooks de figuras corren en CPU a partir de los resultados guardados.

REFERENCIAS

- [1] J. L. Elman, "Finding structure in time," *Cogn. Sci.*, vol. 14, no. 2, pp. 179–211, 1990.
- [2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [3] K. Cho *et al.*, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1724–1734.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [5] G. Larvor, L. Berthomier, V. Chabot, B. Le Pape, B. Pradel, and L. Perez, "MeteoNet, an open reference weather dataset by Météo-France," Météo-France, 2020. [En línea]. Disponible en: <https://meteonet.umr-cnrm.fr/>
- [6] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [7] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.
- [9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510–4520.
- [10] D. S. Velasco Munar and A. Barraza, "Taller 1 – Redes neuronales recurrentes y convolucionales," repositorio de GitHub, 2026. [En línea]. Disponible en: <https://github.com/DANIEL-VELASCO/taller1-deep-learning>
- [11] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, Mar. 1994.
- [12] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
- [13] Z. Karevan and J. A. K. Suykens, "Transductive LSTM for time-series prediction: An application to weather forecasting," *Neural Netw.*, vol. 125, pp. 1–9, 2020, doi: 10.1016/j.neunet.2019.12.030.
- [14] P. Hewage *et al.*, "Temporal convolutional neural (TCN) network for an effective weather forecasting using time-series data from the local weather station," *Soft Comput.*, vol. 24, pp. 16453–16482, 2020, doi: 10.1007/s00500-020-04954-0.
- [15] J. W. Hou, Y. J. Wang, J. Zhou, and Q. Tian, "Prediction of hourly air temperature based on CNN–LSTM," *Geomatics, Natural Hazards Risk*, vol. 13, no. 1, pp. 1962–1986, 2022, doi: 10.1080/19475705.2022.2102942.
- [16] TensorFlow, "Time series forecasting," *TensorFlow Core Tutorials*. [En línea]. Disponible en: https://www.tensorflow.org/tutorials/structured_data/time_series
- [17] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [En línea]. Disponible en: <https://otexts.com/fpp3/>
- [18] Zalando Research, "Fashion-MNIST: Benchmark," repositorio de GitHub. [En línea]. Disponible en: <https://github.com/zalando-research/fashion-mnist> (consultado: 26 de septiembre de 2026).
- [19] S. Bhatnagar, D. Ghosal, and M. H. Kolekar, "Classification of fashion article images using convolutional neural networks," in *Proc. 4th Int. Conf. Image Inf. Process. (ICIIP)*, 2017.
- [20] O. Nocentini, J. Kim, M. Z. Bashir, and F. Cavallo, "Image classification using multiple convolutional neural networks on the Fashion-MNIST dataset," *Sensors*, vol. 22, no. 23, Art. no. 9544, 2022, doi: 10.3390/s22239544.
- [21] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, 2014, pp. 3320–3328.
- [22] S. Kornblith, J. Shlens, and Q. V. Le, "Do better ImageNet models transfer better?," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 2661–2671.
- [23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [24] L. Prechelt, "Early stopping—But when?," in *Neural Networks: Tricks of the Trade* (Lecture Notes in Computer Science, vol. 1524), G. B. Orr and K.-R. Müller, Eds. Berlin, Germany: Springer, 1998, pp. 55–69.
- [25] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [26] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2016, pp. 265–283.
- [27] J. El-Shawa, E. Bagheri, S. A. Kocak, and Y. Mohsenzadeh, "A graph neural network approach for localized and high-resolution temperature forecasting," 2025, *arXiv:2512.00546*.
- [28] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.
- [29] TensorFlow, "Transfer learning and fine-tuning," *TensorFlow Core Tutorials*. [En línea]. Disponible en: https://www.tensorflow.org/tutorials/images/transfer_learning
- [30] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.
- [31] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive label errors in test sets destabilize machine learning benchmarks," in *Proc. NeurIPS Datasets Benchmarks Track*, 2021.
- [33] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 10096–10106.